FE524 Final Project Report

High Speed Financial Document RAG System

GitHub Link

https://github.com/PradHolla/High-Speed-Finance-RAG

Team Members

1. Pradhyumna Nagaraja Holla (Core Architect and Data Pipeline)
2. Pallavi Maralla Satish (Prompt Engineering and Generation)
3. Pratheek Prakash (Retrieval Evaluation, and shared Generation Evaluation with LLM as Judge)
4. Aditya Nagaraj (Generation Evaluation with LLM as Judge)

Abstract

This project builds a Retrieval Augmented Generation system for financial document analysis using SEC filings. The system ingests filing PDFs, creates embeddings, stores chunks in a vector database, retrieves relevant evidence for user questions, and generates answers with source citations. We evaluate the system in two stages. First, retrieval quality is measured using Recall at k, Context Precision, and MRR. Second, generation quality is measured with an LLM as Judge framework that scores factual accuracy, citation quality, and hallucination behavior. Current results show strong generation quality and stable judge reliability, while retrieval precision remains the main improvement area.

1. Introduction

Analysts spend a lot of time manually reading 10 K and 10 Q filings to collect figures, compare companies, and track risks. That process is slow and inconsistent. The goal of this project is to provide a practical assistant that can answer natural language finance questions directly from filings and cite the exact source pages used in the answer.

The target behavior is straightforward. The system should retrieve evidence first, generate grounded answers only from retrieved content, cite sources clearly, and support repeatable evaluation.

2. Data and Task Definition

The current implementation uses Microsoft fiscal year 2025 filing data from `10-K.pdf` for ingestion and testing.

Two golden datasets are used for evaluation. 1. `data/golden_dataset_pratheek_25.jsonl` with 25 questions (IDs `001` through `025`) 2. `data/golden_dataset_aditya_25.jsonl` with 25 questions (IDs `021` through `045`)

Question generation was LLM assisted using GPT 5.4 and Sonnet 4.6, then converted into JSONL benchmark rows. Each record includes a question, expected answer or required facts, expected source pages, and retrieval keywords after validation against filing evidence.

3. Methodology

3.1 Retrieval Augmented Pipeline

The pipeline has five stages. 1. PDF parsing and semantic chunk construction in `ingestion.py` 2. Embedding generation with Bedrock Titan embeddings 3. Vector storage and search in Qdrant 4. Prompt based answer generation in `retrieval_engine.py` 5. Automated evaluation using `eval_retrieval.py` and `eval_generation.py`

Chunks include metadata fields such as company, filing year, filing type, and page span. This metadata supports filtered retrieval and cleaner source tracing.

3.2 Prompting Strategies

Three generation strategies are implemented. 1. `standard` for grounded QA with citation constraints 2. `comparison` for structured comparative analysis 3. `extraction` for strict JSON style output

The extraction path is validated in code so malformed JSON outputs are rejected early.

3.3 LLM as Judge Reliability Design

The LLM as Judge flow is shared work between Pratheek and Aditya. The script validates judge output in a strict way to avoid silent failures.

Reliability controls include: 1. JSON object extraction from noisy model text 2. Exact key and type validation 3. Numeric coercion and bounds checks 4. Repair pass for malformed judge outputs 5. Judge status telemetry with `ok`, `repaired`, and `failed`

4. Experimental Setup

4.1 Retrieval Evaluation

Retrieval evaluation is implemented in `eval_retrieval.py` using these metrics. 1. Recall at k 2. Context Precision 3. Mean Reciprocal Rank

Chunk relevance is determined by expected page overlap and retrieval keyword matching.

4.2 Generation Evaluation (LLM as Judge)

Generation evaluation is implemented in `eval_generation.py` and co owned by Pratheek and Aditya.

The judge returns: 1. factual_accuracy on a 1 to 5 scale 2. citation_quality on a 1 to 5 scale 3. hallucination as boolean 4. missing_facts as list 5. unsupported_claims as list 6. final_score on a 0 to 100 scale

Full runs were executed on both 25 question datasets with `top_k=5` and `strategy=standard`.

5. Results

5.1 Retrieval Results

Source artifact: `artifacts/retrieval_eval_45.json`

Results: 1. Questions evaluated: 45 2. Recall@5: 0.8889 3. Context Precision: 0.5156 4. MRR: 0.7889

Interpretation: the retriever usually finds relevant context, but the top k set still contains enough noise to lower precision.

5.2 Generation Results

Source artifact: `artifacts/generation_eval_aditya25_full_rerun.json`

Results: 1. Questions evaluated: 25 2. Average final score: 91.6 3. Average factual accuracy: 4.76 out of 5 4. Average citation quality: 4.8 out of 5 5. Hallucination rate: 0.04 6. Judge success, repair, failure: 1.0, 0.0, 0.0

Source artifact: `artifacts/generation_eval_pratheek25_full.json`

Results: 1. Questions evaluated: 25 2. Average final score: 91.2 3. Average factual accuracy: 4.72 out of 5 4. Average citation quality: 4.8 out of 5 5. Hallucination rate: 0.0 6. Judge success, repair, failure: 1.0, 0.0, 0.0

Interpretation: generation quality is consistently strong on both sets, and judge reliability controls are stable.

6. Role Wise Contributions

Pradhyumna

Set up the core engineering stack using `uv` lock based dependency management, Qdrant on Docker, Bedrock model access, and the FastAPI API layer consumed by evaluation scripts and the frontend. Implemented PDF ingestion and retrieval foundations including page-aware semantic chunking, Titan embedding generation, Qdrant upsert and query flow, metadata filters, and Docker runtime fixes for stable local deployment.

Pallavi

Designed and integrated prompt logic for `standard`, `comparison`, and `extraction` modes, with clear grounding rules so factual claims are tied to retrieved evidence and citations. Built stricter response-shaping patterns for extraction outputs so structured answers are consistent, machine-usable, and compatible with automated benchmark scoring.

Pratheek

Created and expanded the retrieval benchmark dataset and implemented retrieval evaluation code that computes Recall@k, Context Precision, and MRR from expected pages and keyword relevance checks. Co-owned LLM-as-Judge evaluation with Aditya, contributed a separate 25-question generation dataset produced via LLM-assisted drafting, and validated benchmark runs for quality and consistency.

Aditya

Co-owned LLM-as-Judge evaluation with Pratheek, including rubric definition, scoring workflow checks, and interpretation of generation benchmark behavior across datasets. Contributed a separate 25-question generation dataset produced via LLM-assisted drafting and implemented judge reliability hardening through strict JSON validation, malformed-output repair handling, and success or repair or failure telemetry.

7. Source Code, Data Files, and Accuracy Artifacts

This section summarizes the implementation assets included for FE524 source code deliverables.

7.1 Core Source Code Files

1. `ingestion.py` for extraction, chunking, metadata assignment, embeddings, and upsert
2. `retrieval_engine.py` for retrieval, filtering, context assembly, and generation strategies
3. `backend_api.py` for API endpoints (`/health`, `/ingest`, `/chat`)
4. `eval_retrieval.py` for retrieval metrics benchmarking
5. `eval_generation.py` for LLM as Judge benchmarking
6. `frontend/src/App.jsx` for chat and ingestion UI

7.2 Data Files

1. `10-K.pdf` as primary filing input
2. `data/golden_dataset_pratheek_25.jsonl`
3. `data/golden_dataset_aditya_25.jsonl`

4. `data/golden_dataset.jsonl` as earlier combined set

7.3 Accuracy Analysis Outputs

1. `artifacts/retrieval_eval.json`
2. `artifacts/retrieval_eval_45.json`
3. `artifacts/generation_eval_aditya25_full_rerun.json`
4. `artifacts/generation_eval_pratheek25_full.json`
5. `artifacts/generation_eval_smoke.json`

7.4 End to End Connection

1. `ingestion.py` pushes chunked embedded data to Qdrant
2. `retrieval_engine.py` performs evidence retrieval and answer generation
3. `backend_api.py` exposes system behavior for UI and API usage
4. Evaluation scripts consume golden sets and write benchmark artifacts

8. Completion Status and Remaining Scope

Completed work: 1. End to end RAG pipeline from ingestion to cited answers 2. Prompt strategy integration in runnable code 3. Retrieval and generation evaluation scripts with saved artifacts 4. Two separate 25 question golden datasets with role ownership 5. Full generation evaluation runs on both datasets

Remaining work: 1. Expand corpus coverage beyond a single filing source 2. Improve retrieval precision from current baseline 3. Define final acceptance thresholds for demo readiness 4. Add short error analysis appendix for low scoring cases

9. Reproducibility Commands

```
uv run eval_retrieval.py --golden-path data/golden_dataset_pratheek_25.jsonl --top-k 5 --out

uv run eval_generation.py --golden-path data/golden_dataset_aditya_25.jsonl --top-k 5 --stra

uv run eval_generation.py --golden-path data/golden_dataset_pratheek_25.jsonl --top-k 5 --st
```